DESPLEGAMENT VERSIÓ ANDROID

DAM: MPOP2

1. INTRODUCCIÓ **3**
2. INSTAL·LAR I CONFIGURAR ANDROID STUDIO **3**
2.1. Instal·lar Android Studio (si no el tenim) **3**
2.2. Configurar SDK Manager **3**
3. CONFIGURAR EMULADOR DE DISPOSITIU **6**
4. EMULAR APLICACIÓ EN VISUAL STUDIO CODE **9**
5. GENERAR VERSIÓ RELEASE **15**
5.1. Afegir “launcher icon” 15
5.2. Signar l’aplicació 17
Crear Upload Key 17
Crear key.properties 18
Configurar Gradle pel build 19
Revisar configuració Gradle per build 20
5.3. Generar l’APK 21
Revisar configuració Gradle per build 21
5.4. Desplegar l’APK 22
Provar en dispositiu físic 22
Instal·lar l’APK en dispositiu 23
5.5. Crear workflow CI/CD 23

1. INTRODUCCIÓ

En aquest document veurem el procediment per generar una aplicació Android correctament signada i que podrem instal·lar en un emulador, en un dispositiu real (mòbil o tablet Android) o bé publicar-ho en una Store com Google Play (per motius de cost de registre en aquesta plataforma no farem aquesta última fase).

2. INSTAL·LAR I CONFIGURAR ANDROID STUDIO

Per desplegar una aplicació localment en versió Android primer haurem de configurar l'entorn d'Android Studio. Per fer-ho seguirem la guia de Flutter:

https://docs.flutter.dev/platform-integration/android/setup

2.1. Instal·lar Android Studio (si no el tenim)

Descarregar des de https://developer.android.com/studio i seguir els passos d'instal·lació: https://developer.android.com/studio/install

Instal·lar:

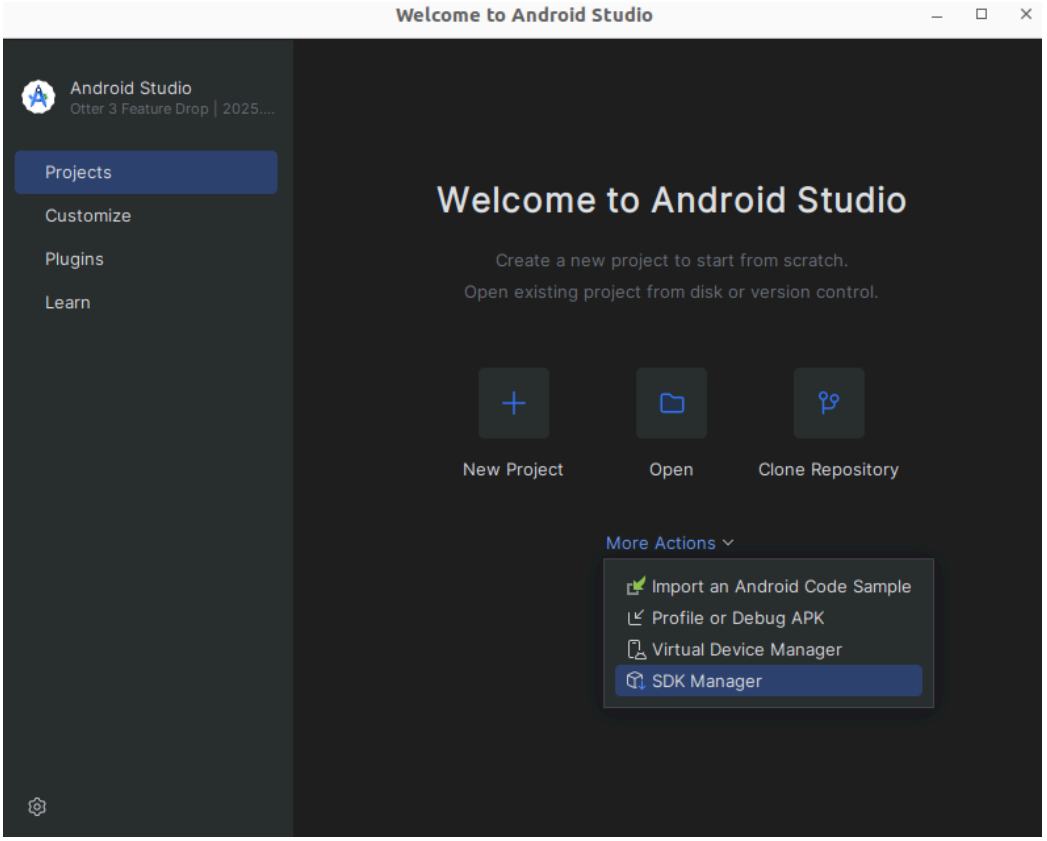
```
sudo tar -xzf android-studio-2025.2.3.9-linux.tar.gz -C
/usr/local/
```

Instal·lar llibreries per 64-bit:

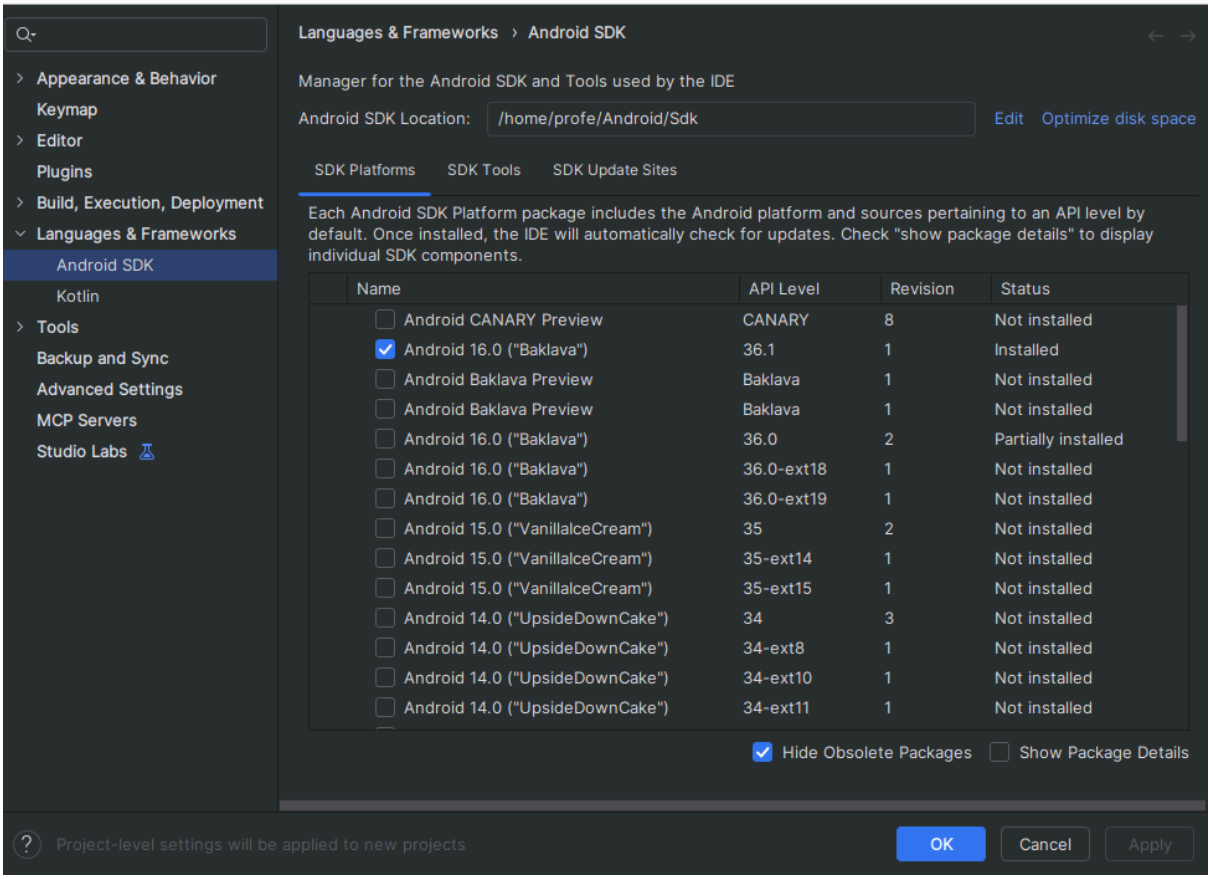
```
sudo apt-get install libc6:i386 libncurses5:i386 libstdc++6:i386
lib32z1 libbz2-1.0:i386
```

2.2. Configurar SDK Manager

Primer haurem d'assegurar que el SDK disposa de tots els elements necessaris. Entrem al SDK Manager:

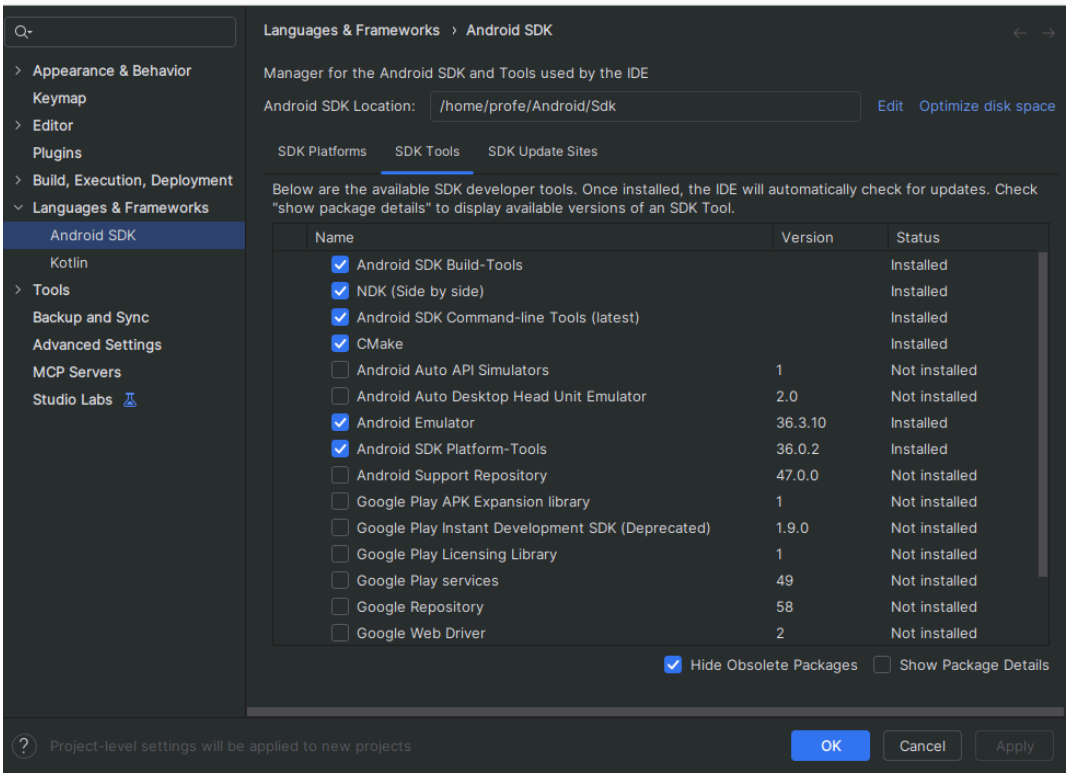


Marcar i instal·lar si és necessari la SDK Platform més recent (API Level més alt per generar aplicacions amb l'última API d'Android):



Marcar i instal·lar si és necessari les diferents SDK Tools:

- Android SDK Build-Tools
- Android SDK Command-line Tools
- Android Emulator
- Android SDK Platform-Tools
- CMake
- NDK (Side by side)



Finalment acceptarem la llicència Android. Flutter ja ens proporciona l'eina per fer-ho per comandes:

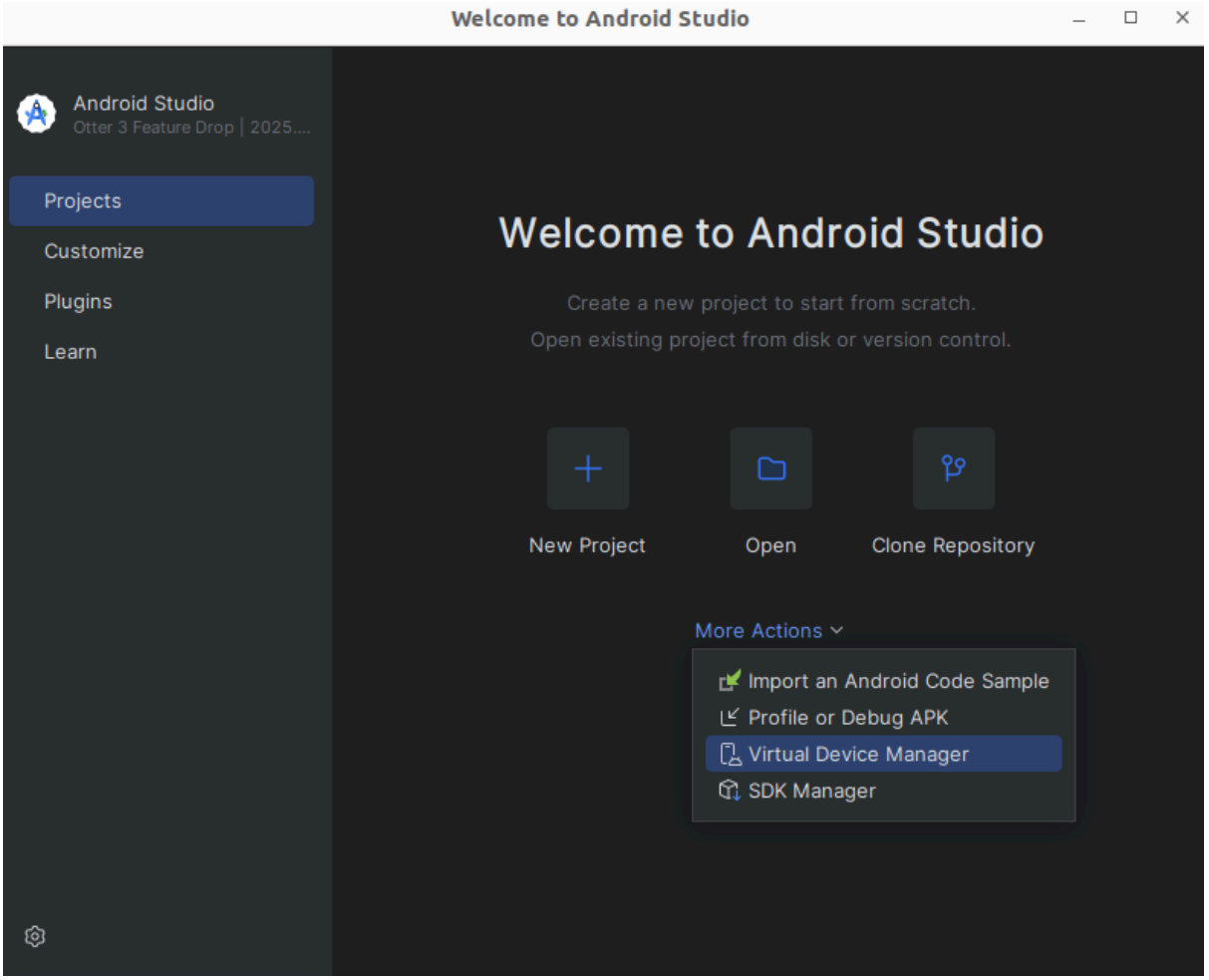
```
flutter doctor --android-licenses
```

3. CONFIGURAR EMULADOR DE DISPOSITIU

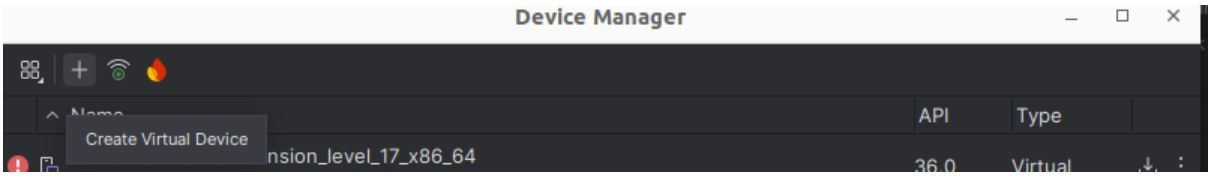
Primer comprovar que tenim l'acceleració VM habilitada (millora rendiment de l'emulador):

```
dgarcia@insebre-com:~/Android/Sdk/emulator $ emulator -accel-check
accel:
0
KVM (version 12) is installed and usable.
accel
```

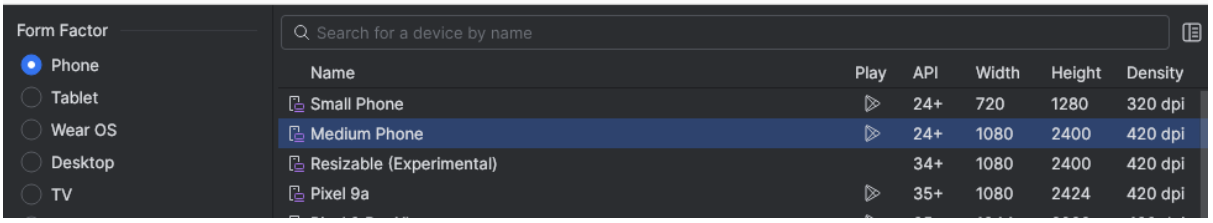
Crear un nou dispositiu virtual accedint al manager d'Android Studio:



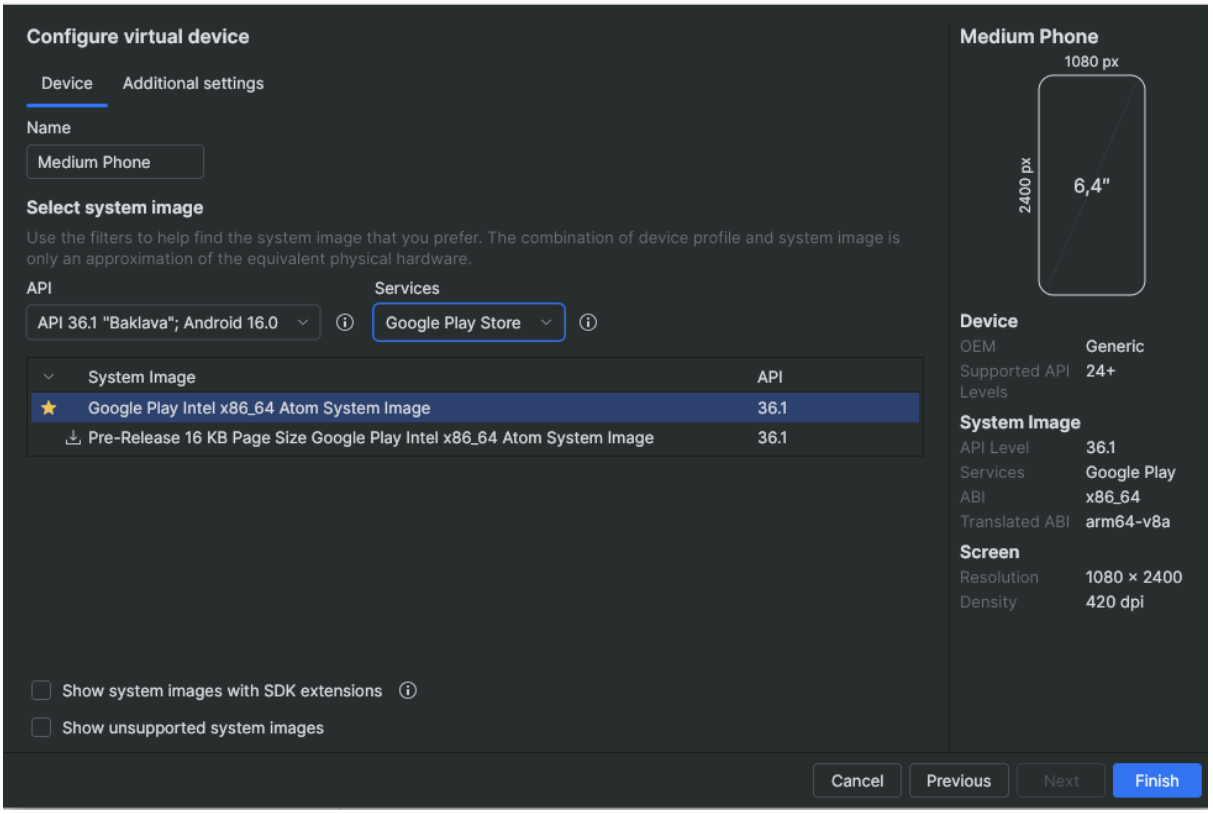
Crear nou dispositiu:



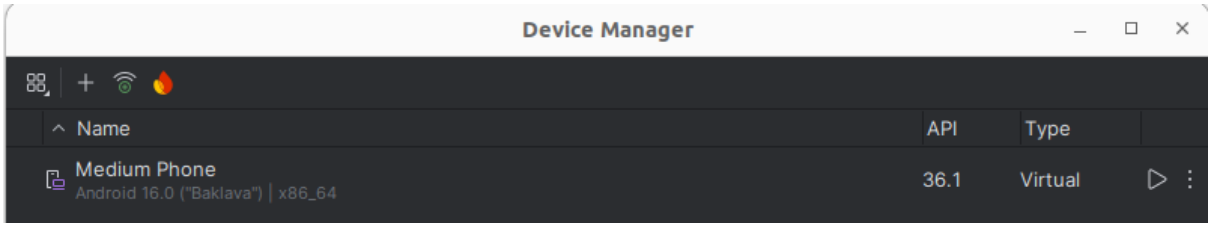
Seleccionar una imatge de dispositiu (mòbil o tablet) escollint la definició que vulguem provar (i cliquem en Next):



Com veiem la plataforma de l'emulador serà x86_64, fet que permetrà un ús més ràpid de l'emulador gràcies a l'acceleració per hardware (kvm). En un dispositiu real, normalment ens trobarem amb una plataforma ARM. Confirmar les opcions seleccionades:

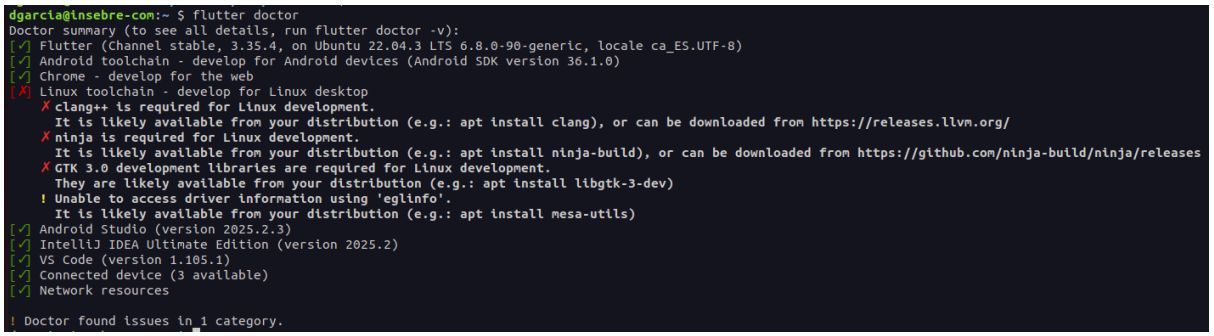


I clicar en “Finish” per descarregar la imatge (si no la tenim) i finalitzar la creació del dispositiu:



Comprovem que la configuració d’Android Studio és correcta executant l’eina de diagnòstic de Flutter:

```
flutter doctor
```



També podem comprovar que els dispositius creats són correctes:

```
flutter emulators && flutter devices
```

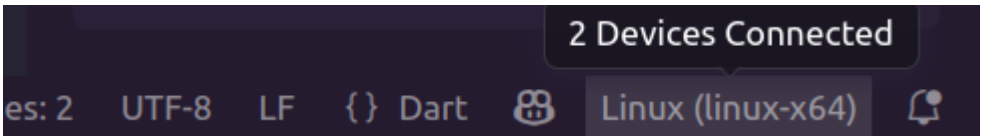
El resultat mostra els emuladors existents i els dispositius disponibles.

```
dgarcia@insebre-com:~ $ flutter emulators && flutter devices
3 available emulators:

Id                                      • Name                                      • Manufacturer • Platform

Medium_Phone                            • Medium Phone                              • Generic      • android
Pixel_3a_API_36_extension_level_17_x86_64 • Pixel_3a_API_36_extension_level_17_x86_64 • Google       • android
Small_Phone                             • Small Phone                               • Generic      • android

To run an emulator, run 'flutter emulators --launch <emulator id>'.
To create a new emulator, run 'flutter emulators --create [--name xyz]'.

You can find more information on managing emulators at the links below:
  https://developer.android.com/studio/run/managing-avds
  https://developer.android.com/studio/command-line/avdmanager
Found 3 connected devices:
  sdk gphone64 x86 64 (mobile) • emulator-5554 • android-x64    • Android 16 (API 36) (emulator)
  Linux (desktop)              • linux         • linux-x64      • Ubuntu 22.04.3 LTS 6.8.0-90-generic
  Chrome (web)                 • chrome        • web-javascript • Google Chrome 144.0.7559.109

Run "flutter emulators" to list and start any available device emulators.

If you expected another device to be detected, please run "flutter doctor" to diagnose potential issues. You may also try increasing the time to wait for connected devices with the "--device-timeout"
flag. Visit https://flutter.dev/setup/ for troubleshooting tips.
```

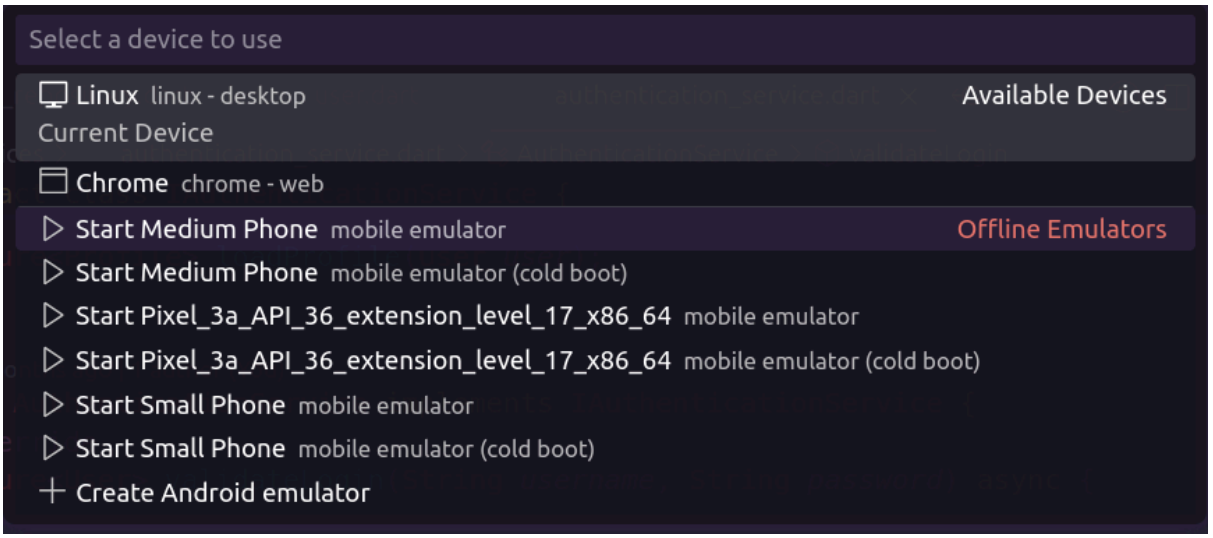
4. EMULAR APLICACIÓ EN VISUAL STUDIO CODE

Farem servir Visual Studio Code per emular, ja que és més lleuger que Android Studio.

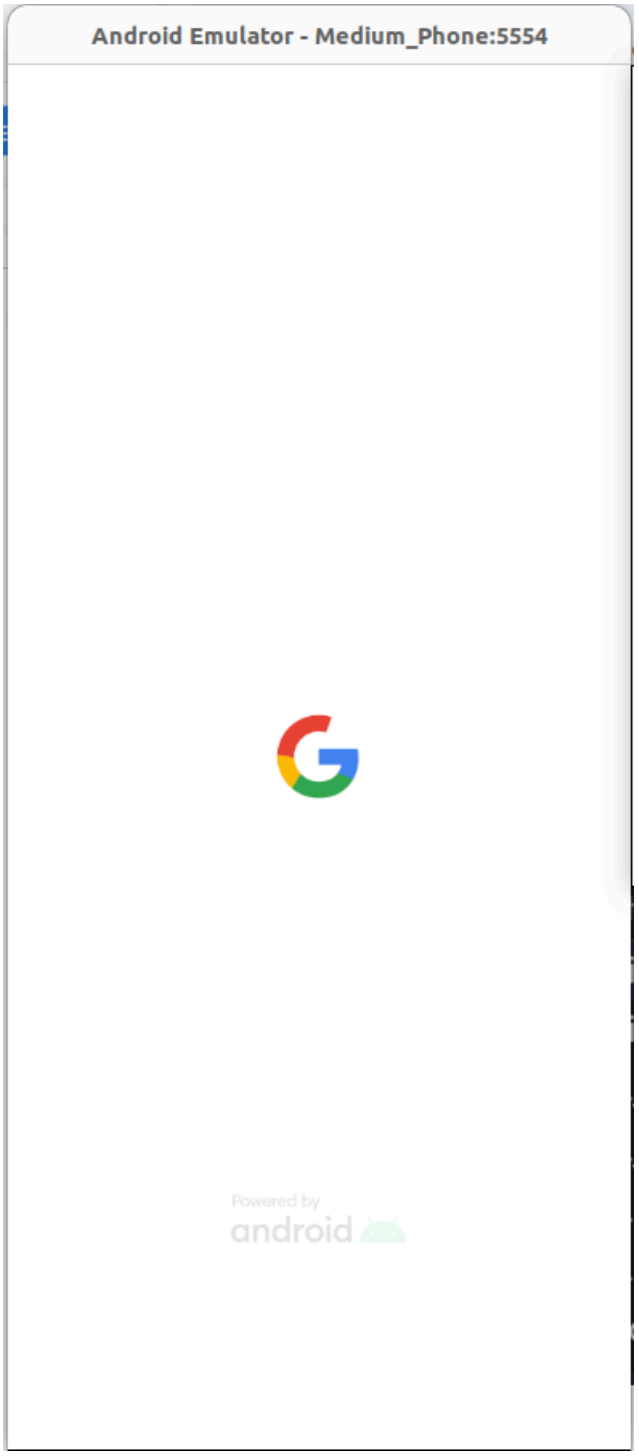
Per emular l'aplicació en un dispositiu mòbil primer seleccionarem el dispositiu a la barra inferior (inicialment apareix Linux (linux-x64) o Chrome):



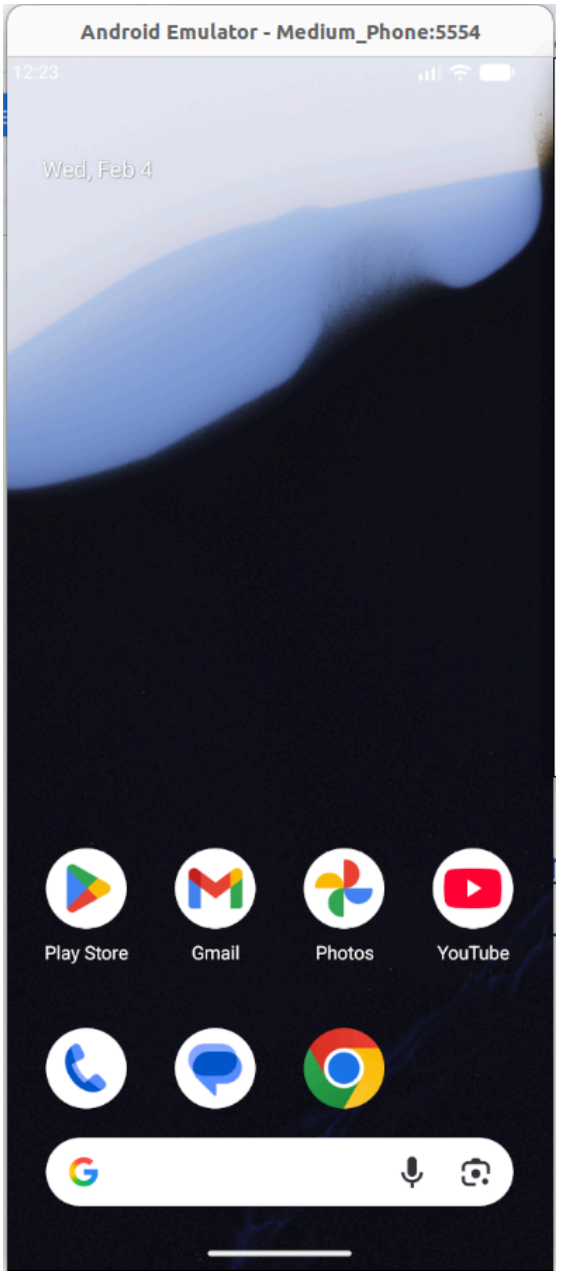
I seleccionem el dispositiu de la llista:



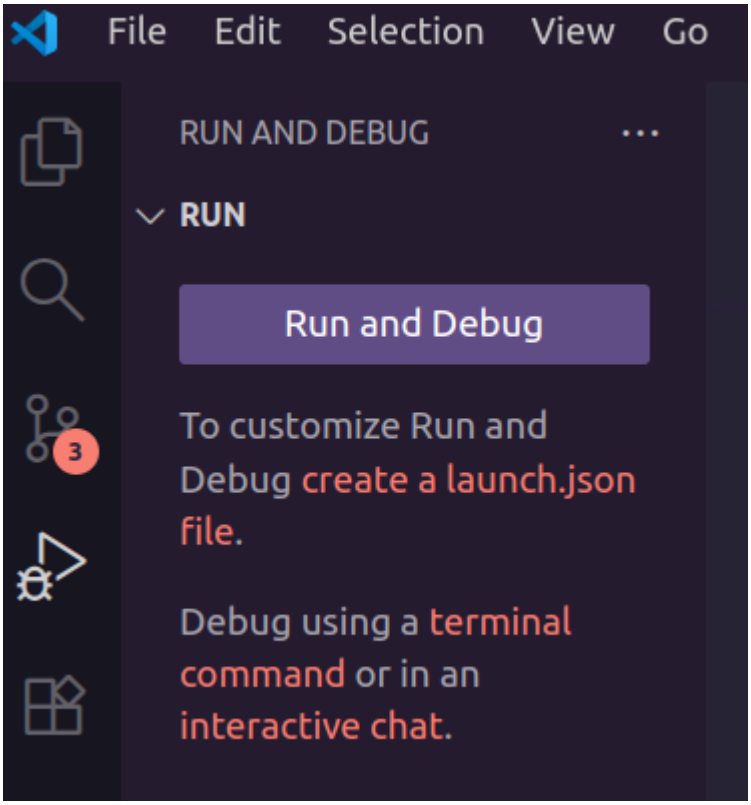
En aquest cas provarem amb Medium Phone. Una vegada seleccionat s'inicia el dispositiu:



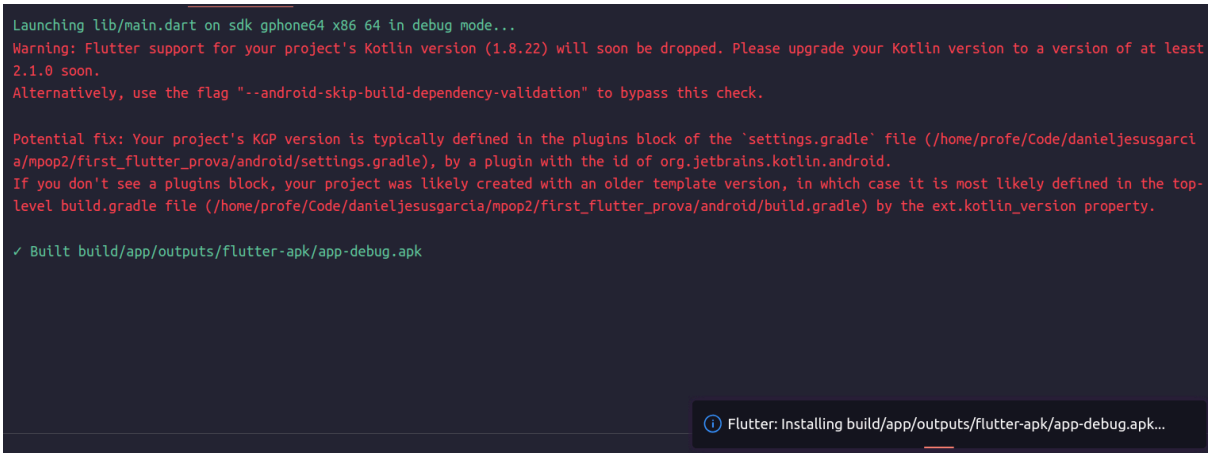
Una vegada iniciat veiem el sistema carregat:



L’aplicació encara no s’està executant. Iniciem l’execució amb “Run and Debug”:

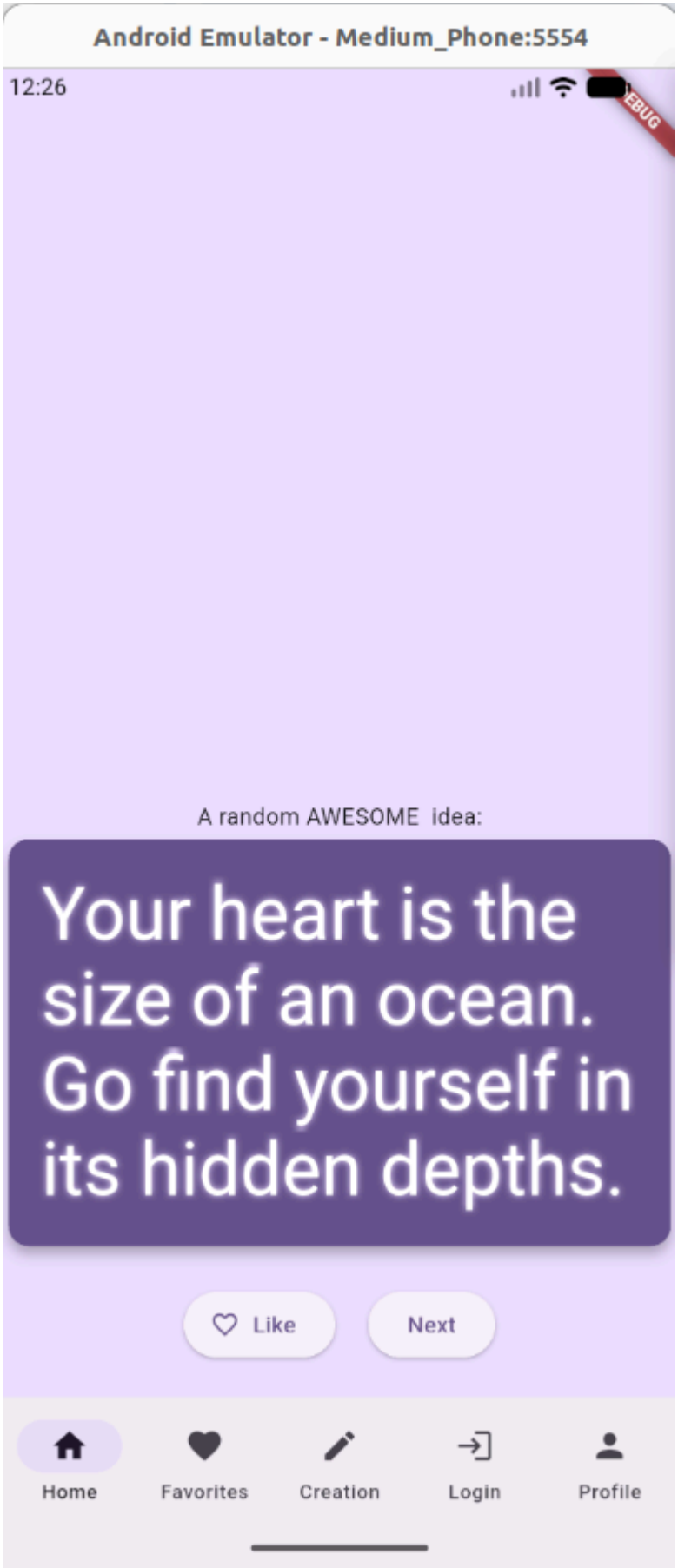


L'aplicació s'inicia en mode "Debug" (no és una versió per desplegar en dispositius reals o publicar en Store):



```
Alternatively, use the flag "--android-skip-build-dependency-validation" to bypass this check.

Potential fix: Your project's KGP version is typically defined in the plugins block of the `settings.gradle` file (/home/profe/Code/danieljesusgarci
a/mpop2/first_flutter_prova/android/settings.gradle), by a plugin with the id of org.jetbrains.kotlin.android.
If you don't see a plugins block, your project was likely created with an older template version, in which case it is most likely defined in the top-
level build.gradle file (/home/profe/Code/danieljesusgarcia/mpop2/first_flutter_prova/android/build.gradle) by the ext.kotlin_version property.

✓ Built build/app/outputs/flutter-apk/app-debug.apk
D/FlutterJNI( 5290): Beginning load of flutter...
D/FlutterJNI( 5290): flutter (null) was loaded normally!
I/flutter ( 5290): [IMPORTANT:flutter/shell/platform/android/android_context_gl_impeller.cc(104)] Using the Impeller rendering backend (OpenGLES).
Connecting to VM Service at ws://127.0.0.1:42987/F7OyK9BDN6g=/ws
Connected to the VM Service.
I/Choreographer( 5290): Skipped 117 frames!  The application may be doing too much work on its main thread.
D/WindowLayoutComponentImpl( 5290): Register WindowLayoutInfoListener on Context=com.example.first_flutter.MainActivity@9e3df3, of which baseContext=
android.app.ContextImpl@8eaf2e5
I/e.first_flutter( 5290): Compiler allocated 5087KB to compile void android.view.ViewRootImpl.performTraversals()
I/Choreographer( 5290): Skipped 44 frames!  The application may be doing too much work on its main thread.
```

L’aplicació iniciada en l’emulador:


Android Emulator - Medium_Phone:5554
12:26
DEBUG
A random AWESOME idea:
Your heart is the size of an ocean. Go find yourself in its hidden depths.
Like
Next
Home
Favorites
Creation
Login
Profile

També podríem haver arrencat l'aplicació en mode debug directament des del terminal:

```
flutter run
```

5. GENERAR VERSIÓ RELEASE

Per generar la versió Release seguirem la documentació a: https://docs.flutter.dev/deployment/android

5.1. Afegir "launcher icon"

Abans de publicar l'aplicació hauríem de configurar la icona de l'aplicació. A nivell de disseny és un element important, ja que identifica l'aplicació en els dispositius. És una marca.

Per aconseguir-ho afegirem alguns elements addicionals al projecte.

Primer afegirem el paquet flutter_launcher_icons al bloc dev_dependencies de pubspec.yaml:

```
flutter_launcher_icons: ^0.14.4
```

I generarem la configuració:

```
flutter pub get
dart run flutter_launcher_icons:generate
```

Com a resultat es genera aquest fitxer:

flutter_launcher_icons.yaml

```yaml
# flutter pub run flutter_launcher_icons
flutter_launcher_icons:
  image_path: "assets/icon/icon.png"

  android: "launcher_icon"
  # image_path_android: "assets/icon/icon.png"
  min_sdk_android: 21 # android min sdk min:16, default 21
  # adaptive_icon_background: "assets/icon/background.png"
  # adaptive_icon_foreground: "assets/icon/foreground.png"
  # adaptive_icon_foreground_inset: 16
  # adaptive_icon_monochrome: "assets/icon/monochrome.png"

  ios: true
  # image_path_ios: "assets/icon/icon.png"
  remove_alpha_ios: true
  # image_path_ios_dark_transparent: "assets/icon/icon_dark.png"
  # image_path_ios_tinted_grayscale: "assets/icon/icon_tinted.png"
  # desaturate_tinted_to_grayscale_ios: true
  # background_color_ios: "#ffffff"

  web:
    generate: true
    image_path: "path/to/image.png"
    background_color: "#hexcode"
    theme_color: "#hexcode"

  windows:
    generate: true
    image_path: "path/to/image.png"
    icon_size: 48 # min:48, max:256, default: 48

  macos:
```

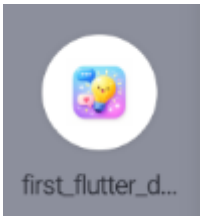
Dipositem una imatge (icona) a la ruta assets/icon/icon.png i actualitzem les altres rutes.

Executem comanda per actualitzar les icones:

```
dart run flutter_launcher_icons
```

```
FLUTTER LAUNCHER ICONS (v0.14.4)

• Creating default icons Android
• Adding a new Android launcher icon
• Overwriting default iOS launcher icon with new ico
Creating Icons for Web...              done
Creating Icons for Windows...          done
Creating Icons for MacOS...            done

✓ Successfully generated launcher icons
```

Si executem l'aplicació en l'emulador trobarem la icona de la nostra aplicació:



5.2. Signar l'aplicació

Seguint la documentació oficial, crearem les claus per signar l'aplicació. És un pas important per la release, ja que la signatura protegeix l'aplicació de suplantacions en futures actualitzacions, que hauran de mantenir la mateixa signatura.

Referència: https://docs.flutter.dev/deployment/android#sign-the-app

Crear Upload Key

La informació més detallada d'aquest pas el trobem a:

https://developer.android.com/studio/publish/app-signing#generate-key

Primer generem una clau destinada a signar les pujades de l'aplicació (upload):

```
keytool -genkeypair \
  -alias upload \
  -keyalg RSA \
  -keysize 2048 \
  -validity 10000 \
  -keystore upload-keystore.jks
```

El procés ens demanarà informació personal per incloure en el certificat que fa la signatura.

```
dgarcia@insebre-com:~/Code/danieljesusgarcia/mpop2/first_flutter_prova $ keytool -genkeypair \
  -alias upload \
  -keyalg RSA \
  -keysize 2048 \
  -validity 10000 \
  -keystore upload-keystore.jks
Enter keystore password:
Re-enter new password:
Enter the distinguished name. Provide a single dot (.) to leave a sub-component empty or press ENTER to use the default value in braces.
What is your first and last name?
  [Unknown]:  Daniel Garcia
What is the name of your organizational unit?
  [Unknown]:  IESEbre
What is the name of your organization?
  [Unknown]:  IESEbre
What is the name of your City or Locality?
  [Unknown]:  Tortosa
What is the name of your State or Province?
  [Unknown]:  Catalunya
What is the two-letter country code for this unit?
  [Unknown]:  ES
Is CN=Daniel Garcia, OU=IESEbre, O=IESEbre, L=Tortosa, ST=Catalunya, C=ES correct?
  [no]:
Enter the distinguished name. Provide a single dot (.) to leave a sub-component empty or press ENTER to use the default value in braces.
What is your first and last name?
  [Daniel Garcia]:
What is the name of your organizational unit?
  [IESEbre]:  Departament Informàtica
What is the name of your organization?
  [IESEbre]:
What is the name of your City or Locality?
  [Tortosa]:
What is the name of your State or Province?
  [Catalunya]:
What is the two-letter country code for this unit?
  [ES]:
Is CN=Daniel Garcia, OU=Departament Informàtica, O=IESEbre, L=Tortosa, ST=Catalunya, C=ES correct?
  [no]:  yes

Generating 2.048 bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 10.000 days
        for: CN=Daniel Garcia, OU=Departament Informàtica, O=IESEbre, L=Tortosa, ST=Catalunya, C=ES
```

Com a resultat tenim un fitxer upload-keystore.jks.

IMPORTANT: Aquest fitxer no s'ha de pujar al repositori i menys si és un repositori públic.

Crear key.properties

Crear fitxer key.properties dins de carpeta "android" per guardar claus i keystore:

```
android > key.properties
1  storePassword=<password-from-previous-step>
2  keyPassword=<password-from-previous-step>
3  keyAlias=upload
4  storeFile=<keystore-file-location>
```

Aquest fitxer i el keystore han de ser privats (no s'ha de pujar a Git)

En aquest tutorial storeFile es troba a:

```
storeFile=/home/profe/Code/danieljesusgarcia/mpop2/first_flutter_prova/
upload-keystore.jks
```

Configurar Gradle pel build

Modificarem el fitxer “android/app/build.gradle.kts” per afegir la configuració que permetrà signar l’aplicació en el moment de fer el build per Release

```
import java.util.Properties
import java.io.FileInputStream

plugins {
   ...
}

val keystoreProperties = Properties()
val keystorePropertiesFile = rootProject.file("key.properties")
if (keystorePropertiesFile.exists()) {

keystoreProperties.load(FileInputStream(keystorePropertiesFile))
}

android {
   ...
}
```

Afegirem també signingConfigs abans de buildtypes:

```
    signingConfigs {
        create("release") {
            keyAlias = keystoreProperties["keyAlias"] as String
            keyPassword = keystoreProperties["keyPassword"] as
String
            storeFile = keystoreProperties["storeFile"]?.let {
file(it) }
            storePassword = keystoreProperties["storePassword"] as
String
        }
```

```
    }
```

Pe defecte la signatura es fa en versió debug sense cap clau. Per la versió Release, hem de canviar buildTypes per signar amb la configuració proporcionada:

```
            // signingConfig = signingConfigs.getByName("debug")
             signingConfig = signingConfigs.getByName("release")
```

En debug l'aplicació permet l'accés a Internet, però en versió Release aquest permís està deshabilitat per defecte. Per permetre-ho (en el nostre cas fem crides a una api externa) afegim l'accés a Internet per versió Release a l'AndroidManifest.xml:

android/app/src/main/AndroidManifest.xml

```
    <uses-permission android:name="android.permission.INTERNET" />
```

Revisar AndroidManifest.xml i actualitzar l'etiqueta de l'aplicació (serà el nom que apareixerà en instal·lar) si s'escau:

```
        android:label="first_flutter_dg_iesebre"
```

Revisar configuració Gradle per build

Cal assignar un identificador únic per l'aplicació, especialment si l'anem a publicar en un Store. Típicament és un domini invers de l'organització. Per exemple:

```
        applicationId = "cat.insebre.first_flutter_dg_iesebre"
```

Convé també revisar les versions Android per adaptar-ho a la compatibilitat requerida:

- compileSdk: The version of the Android SDK used to compile the app.
- minSdk: The minimum Android version that the app supports.
- targetSdk: The Android version the app is designed and tested to run on.

Per evitar que actualitzacions de Flutter modifiquin aquests valors, s'hauria de modificar pel valor "hardcoded".

5.3. Generar l'APK

Per publicar en Google Play el format d'aplicació preferit és "appbundle" (conté diferents apk en funció de la plataforma destí). Però de moment generarem una apk

Per defecte faríem:

```
flutter build apk --release
```

Però ocuparia massa espai. Per separar els apk per arquitectura farem:

```
flutter build apk --split-per-abi
```

Ens genera diferents apk, cadascun per una arquitectura concreta:

```
djgarcia@Zen2024Ubuntu:~/Code/mpopt2/first_flutter$ flutter build apk --split-per-abi
Resolving dependencies...
Downloading packages...
  characters 1.4.0 (1.4.1 available)
  http 1.5.0 (1.6.0 available)
  lints 6.0.0 (6.1.0 available)
  matcher 0.12.17 (0.12.18 available)
  material_color_utilities 0.11.1 (0.13.0 available)
  meta 1.16.0 (1.18.1 available)
  test_api 0.7.6 (0.7.9 available)
Got dependencies!
7 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
Warning: Flutter support for your project's Kotlin version (1.8.22) will soon be dropped. Please upgrade your Kotlin version to a version of at least 2.1.0 soon.
Alternatively, use the flag "--android-skip-build-dependency-validation" to bypass this check.

Potential fix: Your project's KGP version is typically defined in the plugins block of the `settings.gradle` file (/home/djgarcia/Code/mpopt2/first_flutter/android/settings.gradle), by
 a plugin with the id of org.jetbrains.kotlin.android.
If you don't see a plugins block, your project was likely created with an older template version, in which case it is most likely defined in the top-level build.gradle file (/home/djga
rcia/Code/mpopt2/first_flutter/android/build.gradle) by the ext.kotlin_version property.

Running Gradle task 'assembleRelease'...                        12,2s
✓ Built build/app/outputs/flutter-apk/app-armeabi-v7a-release.apk (13.7MB)
✓ Built build/app/outputs/flutter-apk/app-arm64-v8a-release.apk (16.3MB)
✓ Built build/app/outputs/flutter-apk/app-x86_64-release.apk (17.4MB)
```

En concret genera aquests 3 APKs:

✓ Built build/app/outputs/flutter-apk/app-armeabi-v7a-release.apk (13.7MB)

✓ Built build/app/outputs/flutter-apk/app-arm64-v8a-release.apk (16.3MB)

✓ Built build/app/outputs/flutter-apk/app-x86_64-release.apk (17.4MB)

Això evita un apk universal més gran, però que contindria versions per arquitectures que no utilitzarà l'usuari (només necessita la del seu dispositiu).

Revisar configuració Gradle per build

```
apksigner verify --verbose
build/app/outputs/flutter-apk/app-arm64-v8a-release.apk
```

```
Verifies
Verified using v1 scheme (JAR signing): false
Verified using v2 scheme (APK Signature Scheme v2): true
Verified using v3 scheme (APK Signature Scheme v3): false
Verified using v4 scheme (APK Signature Scheme v4): false
Verified for SourceStamp: false
Number of signers: 1
```

La verificació amb apksigner confirma que l'APK està signat correctament mitjançant l'esquema V2, que és el mecanisme recomanat en versions modernes d'Android.

5.4. Desplegar l'APK

Finalment instal·lem la APK pel nostre sistema. Les proves realitzades s'han fet en un Xiaomi Note 14 d'arquitectura ARM.

Provar en dispositiu físic

Primer haurem d'activar les opcions de desenvolupador al mòbil:

- Ajustos → About phone
- Toca 7 cops Build number
- Torna enrere → Developer options / Configuracions Addicionals
- Activa USB debugging
- Activa USB Install

A continuació connectem el mòbil per USB. Comprovem que detecta el dispositiu amb adb o Flutter:

```
adb devices
flutter devices
```

Al mòbil haurem d'acceptar la connexió amb una clau RSA

Instal·lar l'APK en dispositiu

Finalment instal·lem l'APK. Si hem generat APKs per plataforma, haurem d'indicar quina instal·lem:

```
flutter install
--use-application-binary=build/app/outputs/flutter-apk/app-arm64-v
8a-release.apk
Installing app-arm64-v8a-release.apk to 24116RACCG...
Uninstalling old version...
Installing
build/app/outputs/flutter-apk/app-arm64-v8a-release.apk...
11,3s
```

Si tenim app-release.apk (que inclou totes les apk), podem fer simplement:

```
flutter install
```

També podríem fer la instal·lació amb l'eina adb:

```
adb install build/app/outputs/flutter-apk/app-release.apk
```

Finalment, podem provar en el mòbil que l'aplicació funciona en el dispositiu físic.

Si no disposem d'un mòbil Android podríem provar d'instal·lar l'aplicació en versió release en un emulador.

5.5. Crear workflow CI/CD

Finalment, per automatitzar el procés de creació del APK, crearem un fitxer yml dins de .github/workflows per la creació del APK:

```yml
name: Flutter Build APK

on:
  push:
    branches: [ "testing" ]

jobs:
  build:
    runs-on: ubuntu-latest
```

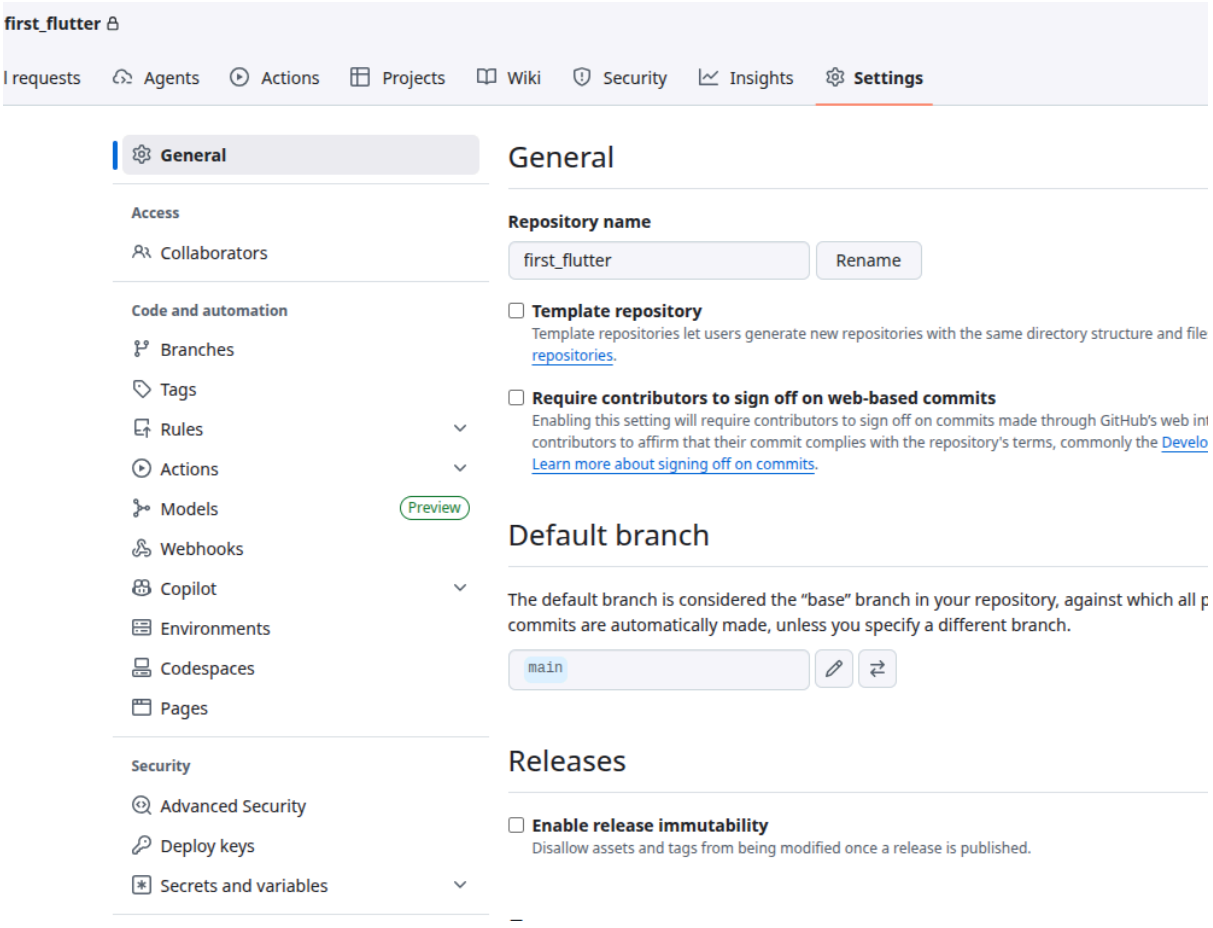
```yaml
    steps:
    - uses: actions/checkout@v4

    # 1. Configurem Java (necessari per a la part d'Android)
    - name: Set up JDK
      uses: actions/setup-java@v4
      with:
        java-version: '17'
        distribution: 'temurin'

    # 2. Instal·lem l'SDK de Flutter
    - name: Set up Flutter
      uses: subosito/flutter-action@v2
      with:
        channel: 'stable'
        cache: true

    # 3. Descarreguem dependències
    - run: flutter pub get

    # 4. Verifiquem el codi (opcional però recomanat)
    - run: flutter test

    # 5. Construïm l'APK
    - run: flutter build apk --release

    # 6. Guardem el fitxer resultant
    - name: Upload APK
      uses: actions/upload-artifact@v4
      with:
        name: flutter-app-release
        path: build/app/outputs/flutter-apk/app-release.apk
```

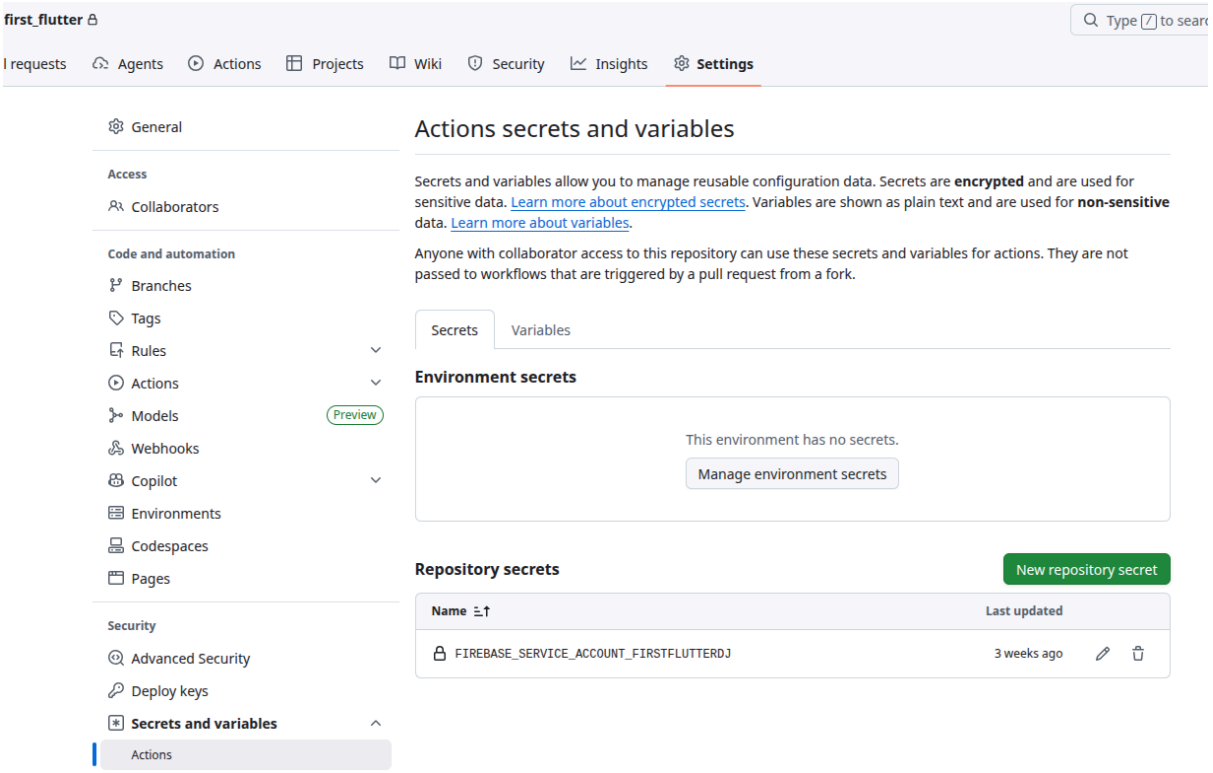
Per poder signar l'aplicació sense compartir en el repositori les claus privades, haurem de configurar les "keys" en GitHub com secrets (fitxer key.properties i upload-keystore.jks).

Primer haurem de generar una cadena a partir del fitxer binari jks (el guardem per fer-lo servir en la creació de secrets):

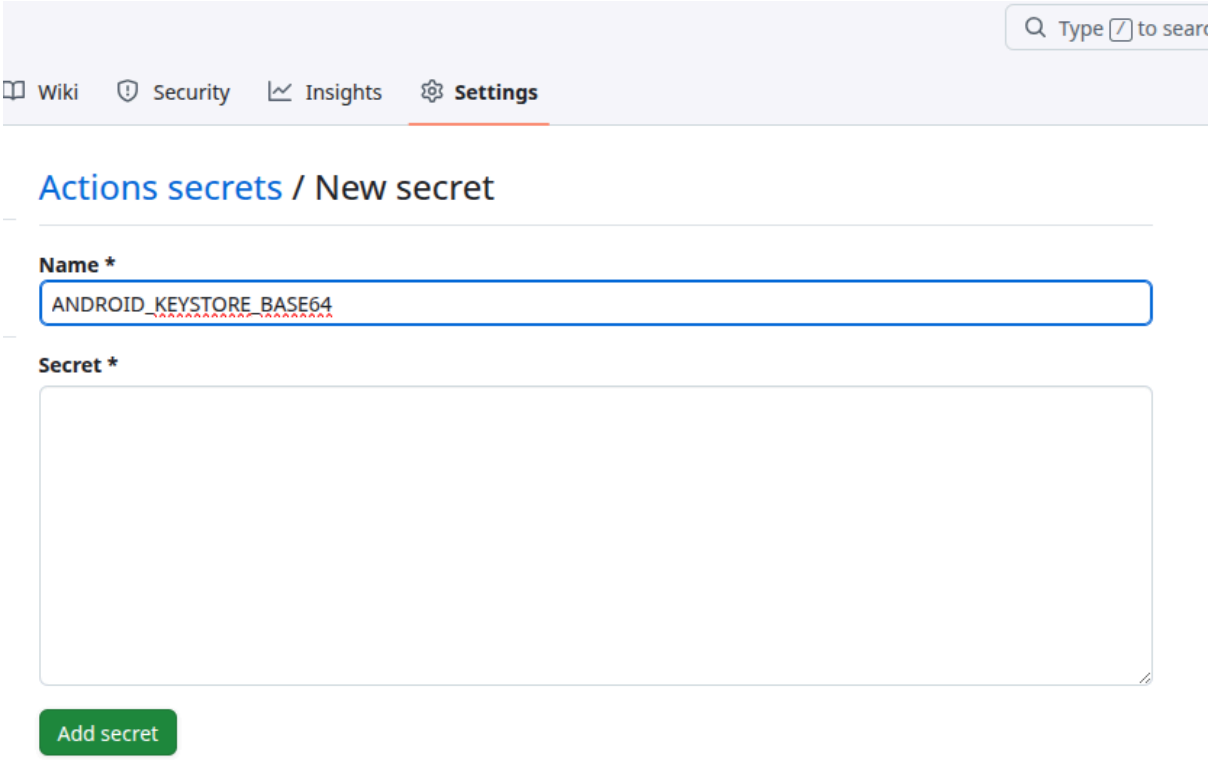
```
openssl base64 -A -in upload-keystore.jks
```

Anirem a Github per configurar els secrets:





Crearem els diferents secrets. Quedaran encriptats. Posteriorment es podran modificar, però no veure el seu valor original:



ANDROID_KEYSTORE_BASE64: Enganxa el text llarg que has copiat abans.

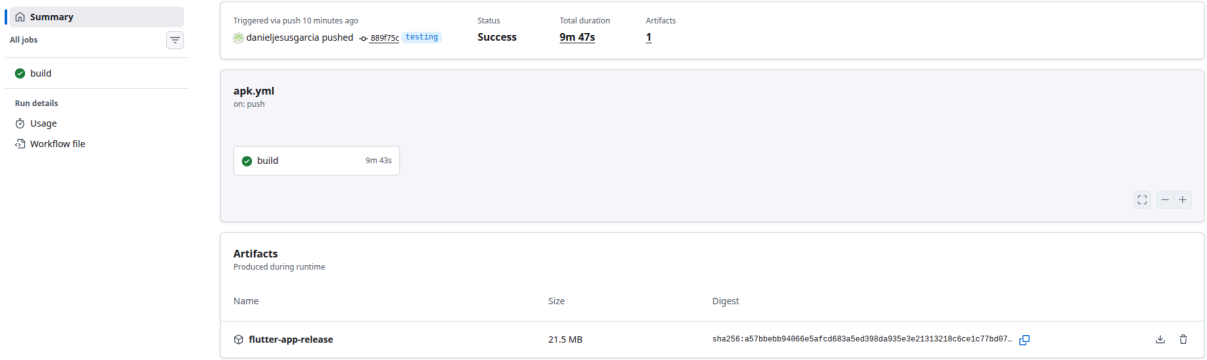
ANDROID_KEYSTORE_PASSWORD: La contrasenya del magatzem.

ANDROID_KEY_PASSWORD: La contrasenya de la clau.

ANDROID_KEY_ALIAS: L'àlies (normalment upload).

Afegim pas previ a fer el “build” per utilitzar els secrets:

```
  - name: Decode Keystore
    run: |
      echo "${{ secrets.ANDROID_KEYSTORE_BASE64 }}" | base64 --decode > android/app/upload-keystore.jks

  - name: Create key.properties
    run: |
      echo "storePassword=${{ secrets.ANDROID_KEYSTORE_PASSWORD }}" > android/key.properties
      echo "keyPassword=${{ secrets.ANDROID_KEY_PASSWORD }}" >> android/key.properties
      echo "keyAlias=${{ secrets.ANDROID_KEY_ALIAS }}" >> android/key.properties
      echo "storeFile=upload-keystore.jks" >> android/key.properties
```

Finalment, afegim el nou fitxer yml al repositori i fem el commit/push.

Automàticament s’executa el workflow en Github. Si finalitza sense errors, disposarem de l’APK com artefacte a descarregar i provar en un dispositiu (físic o emulador).